

DPI343

Computer Organization and Assembly Language

Lecture 2

Lectures - prof. em. U. Sukovskis

Labs for local students - assoc. prof. P. Rusakovs

Labs for foreign students - assoc. prof. G. Alksnis

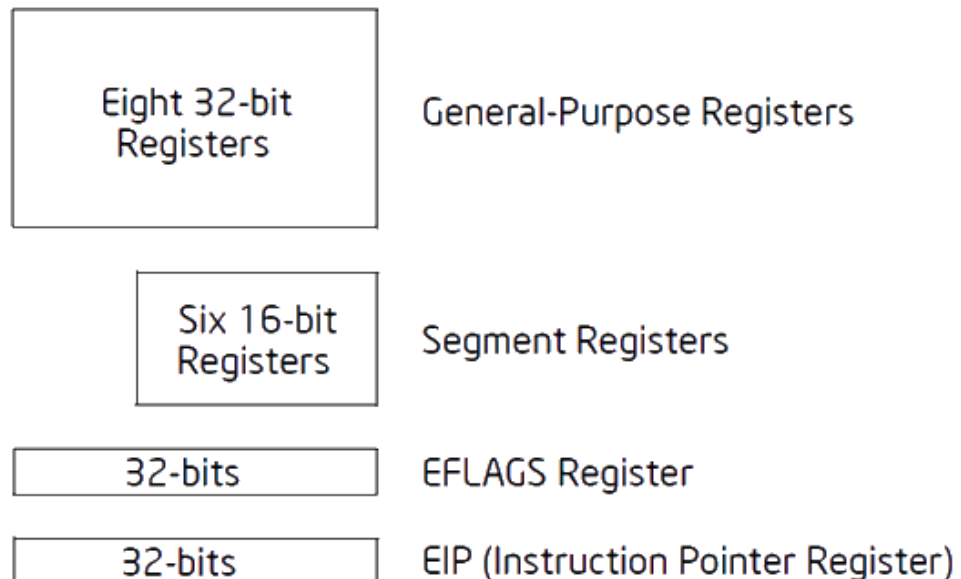
Autumn 2025



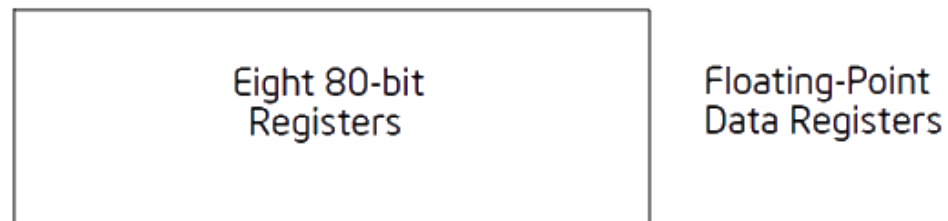
RTU
DATORZINĀTNES,
INFORMĀCIJAS
TEHNOLOĢIJAS
UN ENERĢĒTIKAS
FAKULTĀTE

IA-32 CPU execution environment (simplified)

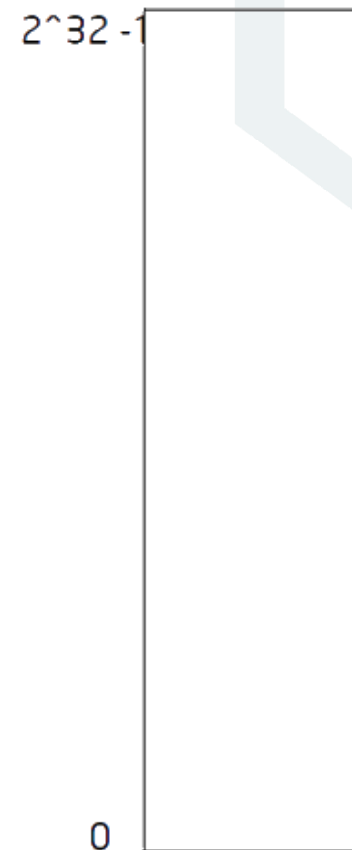
Basic Program Execution Registers



FPU Registers



Address Space*



IA-32 general purpose registers

31	16 15	8 7	0	16-bit	32-bit
	AH	AL		AX	EAX
	BH	BL		BX	EBX
	CH	CL		CX	ECX
	DH	DL		DX	EDX
	BP				EBP
	SI				ESI
	DI				EDI
	SP				ESP

General purpose register mapping



Example:

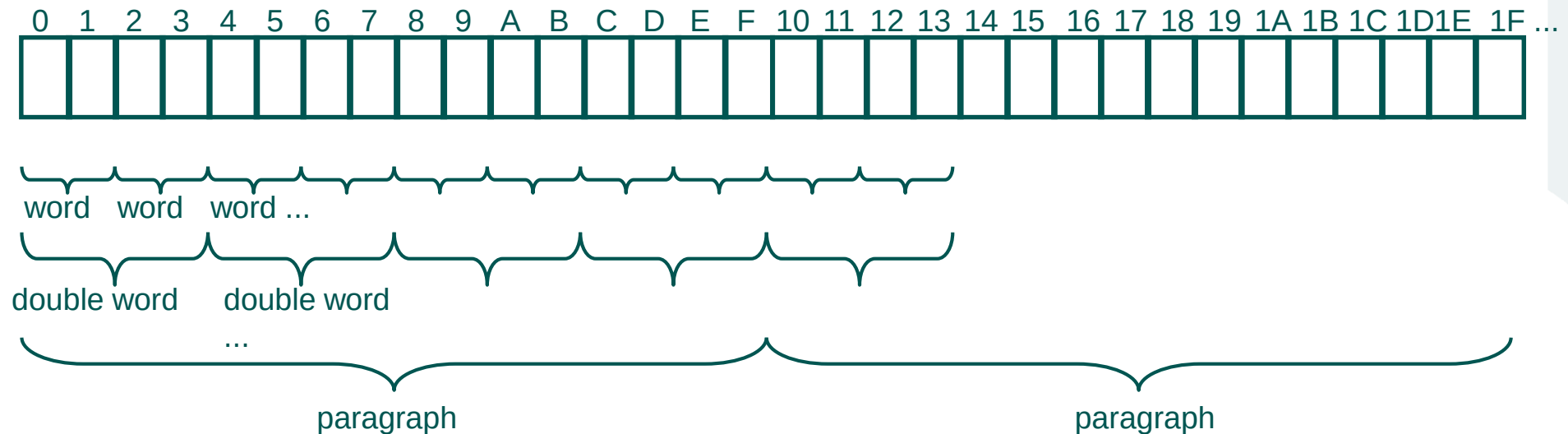
- Register EAX (binary) :

0000'0000'0000'0000'0001'0100'1100'0011

- EAX (in hex) = ? **000014C3** **decimal $1*4096+4*256+12*16+3 = 5315$**
- AX (in hex) = ? **14C3**
- AH (in hex) = ? **14** **decimal $1*16 + 4 = 20$**
- AL (in hex) = ? **C3** **unsigned value is decimal 195**
 signed value is decimal -61

Computer memory addressing

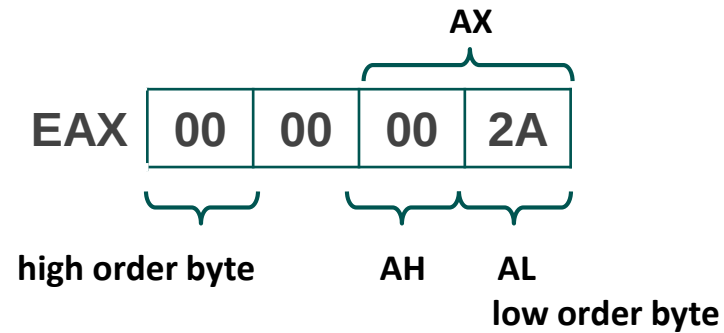
- RAM
- ROM
- Virtual memory
- Memory units
 - Bit, Byte, Word, Doubleword, Quadword, Paragraph



Representation of integer values

Copy integer value 42 (decimal) to the register EAX

```
mov  eax, 42
```

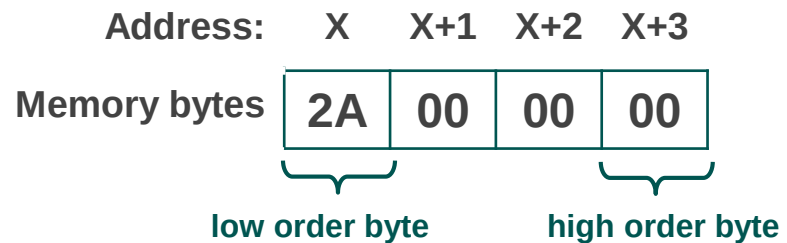


Copy value from the register EAX to the memory doubleword X

```
X  dword 0
```

...

```
mov  X, eax
```

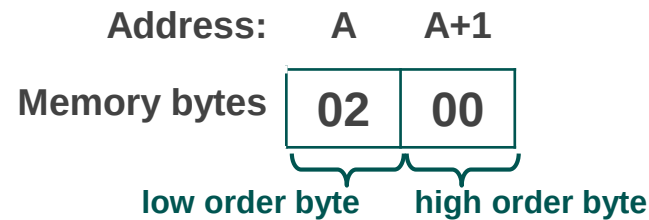


Representation of integer values

Define memory field containing value 2

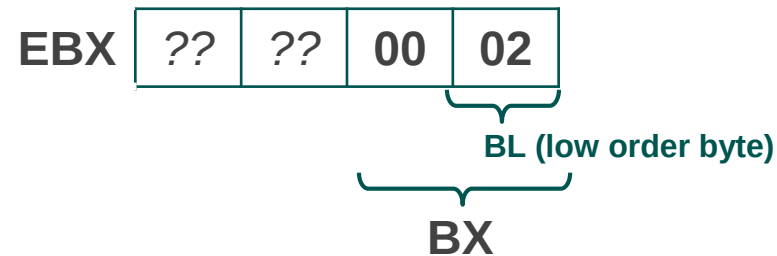
A word 2

Name A represents address of the first memory byte of this memory field with the smallest address and hence this is low order byte of this numeric value.



Copy two bytes from memory address A to the register BX

mov bx, A



Add -4 to the register BX

add bx, -4



Special roles of CPU registers

General purpose registers

- EAX — **Accumulator** for operands and results data
- EBX — Pointer to **base** of data in the memory data segment
- ECX — **Counter** for loop and string operations
- EDX — **Data** for multiplication/division, I/O pointer

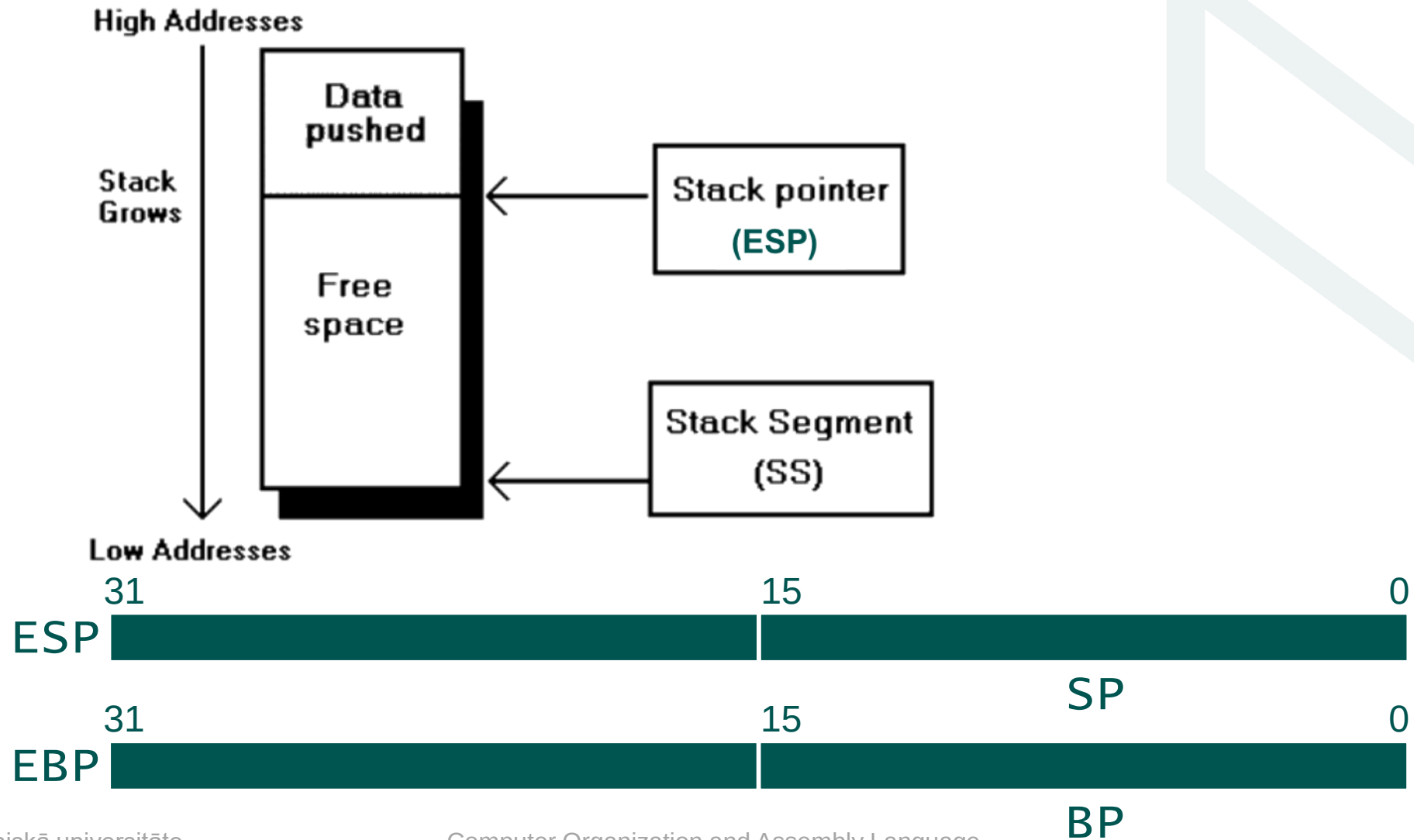
Index registers

- ESI — Pointer to data in the memory data segment; **source index** for string operations
- EDI — Pointer to data (or destination) in the memory data segment; **destination index** for string operations

Stack registers

- ESP — **Stack pointer** (in the stack segment)
- EBP — **Base pointer** for the stack operations in the stack segment

Stack and stack registers



CPU execution status

Flags register

- Flags register is a CPU register where bits represent characteristics of the results of operations and the state of the CPU
- Layout of flags register – some most important flags

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
				OF	DF	IF		SF	ZF				PF		CF

CF	Carry	Carry or borrow to destination operand
PF	Parity	Even parity in low-order byte of result
ZF	Zero	Result equal to 0
SF	Sign	Result has negative sign
IF	Interrupt	Enables interrupts (disables if 0)
DF	Direction	String operations decrement the pointer if 1, increment if 0
OF	Overflow	Result too large or small for destination

Instructions executed by CPU can change the values of flags depending on the result of the instruction. For example, if result of an arithmetic operation is 0 then ZF is set to 1.

Flags also can be set to 1 or cleared to 0 using specific instructions, for example instruction STD sets direction flag to 1 and CLD clears it to 0. We'll learn more about flags during the course.

Instruction pointer register EIP

- Contains memory location of the next instruction to be executed by CPU



- This register cannot be specified directly in assembly instructions, but value of the EIP register can be changed by control transfer instructions like JMP, CALL, RET etc.

Computer memory addressing.

Segment registers

Each of the CPU segment register is associated with one of three types of memory storage used for **code**, **data**, or **stack**.

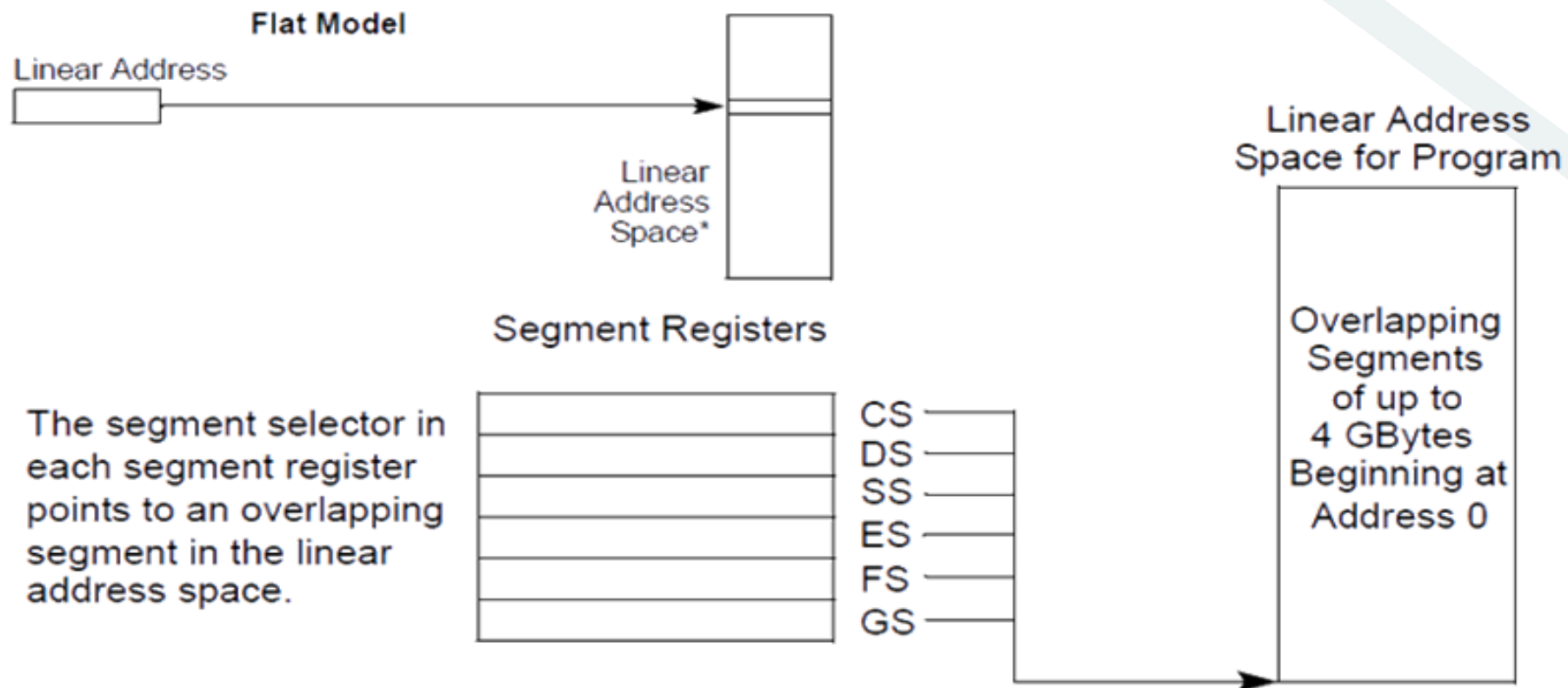
- The CS register contains the segment selector for the **code segment**
- The DS, ES, FS, and GS registers point to four **data segments**.
- The SS register contains the segment selector for the **stack segment**



Memory addressing models.

IA-32 flat model

FLAT memory model has a single segment that contains both code and data. Can be used on a 386 or higher CPU. FLAT memory model works only in address offsets and has a range of 4 gigabytes. This makes assembler easier to write and the code is generally faster. All segment registers are automatically set to the same value.



Memory addressing models.

Real address model (16-bit mode 8086)

Memory address consists of segment and offset components:

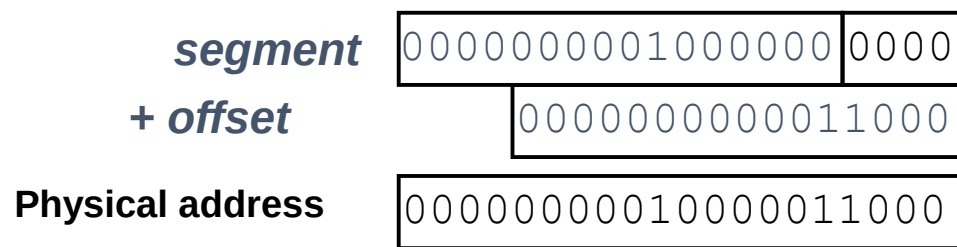
segment : offset

One segment takes up to 64KiB

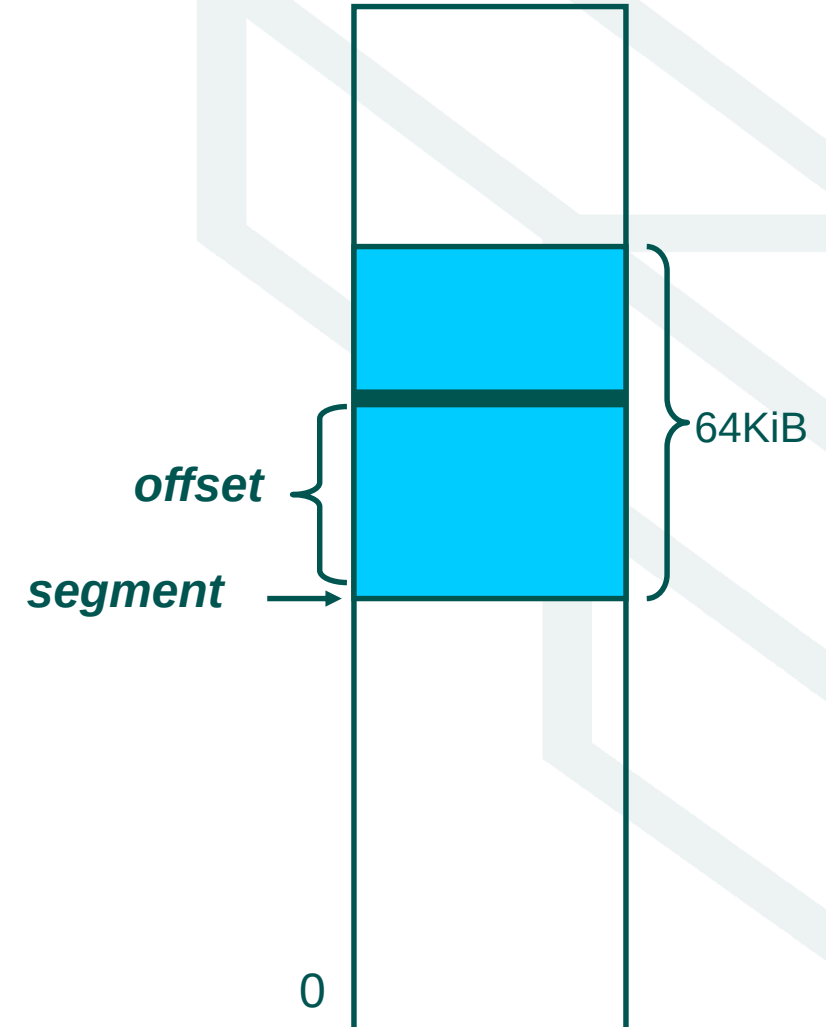
Segments begin at paragraph boundary

Multiple segments can overlap

Physical address = *segment* * 16 + *offset*



Real addressing mode will be used in the second part of the course.



Simple Assembly instructions

General purpose instructions.

MOVE

- The source *operand2* can be:
 - an immediate value;
 - general-purpose register;
 - segment register;
 - memory location.
- The destination *operand1* can be:
 - a general-purpose register;
 - segment register;
 - memory location.
- If one operand is a segment register the other cannot be segment register.
- Both operands must be of the same size (byte, word, doubleword, quadword).
- Execution:
 operand1 = operand2;



General purpose instructions.

MOVSX and MOVZX

- **MOVSX** and **MOVZX** are special versions of the **MOV** instruction that perform sign extension or zero extension from the source to the destination.
This is the only instruction that allows the source and destination to be different sizes (and in fact, they must be different sizes).
- **MOVSX** moves and **sign-extends** the value of the source operand to the destination register. Use MOVSX to copy a signed 8-bit or 16-bit source operand to a larger 16-bit or 32-bit destination register.
- **MOVZX** moves and **zero-extends** the value of the source operand to the destination register. Use MOVZX to copy an unsigned 8-bit or 16-bit source operand to a larger 16-bit or 32-bit destination register.

N1 byte -2 ; one byte contains FE

movsx eax, N1 ; the EAX register will contain FFFFFFFE

movzx ebx, N1 ; the EBX register will contain 000000FE

General purpose instructions.

XCHG

XCHG operand1, operand2

- Exchanges the values of the source and destination operands
- The *operand1* is a register.
- The *operand2* can be a register, or a memory location.
- To swap values of EAX and EBX:
 xchg eax, ebx

Arithmetic instructions.

INC and DEC

INC operand

DEC operand

- Increment adds (decrement subtracts) 1 to (from) the destination operand. The destination operand can be a register or a memory location.
- Execution :
 - For increment: $\text{operand} += 1;$
 - For decrement: $\text{operand} -= 1;$

Arithmetic instructions.

NEG

NEG operand

- The *operand* can be located in a general-purpose register or a memory location.
- Execution :
 $\text{operand} = -\text{operand};$

Arithmetic instructions.

ADD

ADD operand1, operand2

- Adds the destination *operand1* with the source *operand2* and stores the result in the *operand1*.
- The *operand1* can be a register or a memory location.
- The *operand2* can be an immediate value, a register, or a memory location.
- Memory locations for both source and destination cannot be used in the same ADD instruction.
- When an immediate value is used as an *operand2*, it is sign-extended to the size of the *operand1*.
- Execution :
$$\text{operand1} = \text{operand1} + \text{operand2};$$

Arithmetic instructions.

SUB

SUB *operand1*, *operand2*

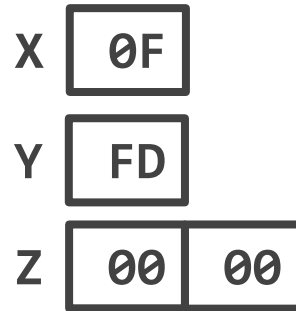
- Subtracts the second *operand2* from the first *operand1* and stores the result in the *operand1*.
- The destination operand can be a register or a memory location.
- The source operand can be an immediate, register, or memory location.
- Memory locations for both source and destination cannot be used in the same SUB instruction.
- When an immediate value is used as an *operand2*, it is sign-extended to the size of the *operand1*.
- Execution :
$$\text{operand1} = \text{operand1} - \text{operand2};$$

Example: Simple calculations

Calculate $X+Y$ where X and Y are byte format integers and store the result into word Z

X byte 15
 Y byte -3
 Z word ?
...

```
mov    al, X
add     al, Y
movsx  ax, al
mov     Z, ax
```



Address: Z $Z+1$



Convert Byte to Word, Word to Doubleword, Doubleword to Quadword

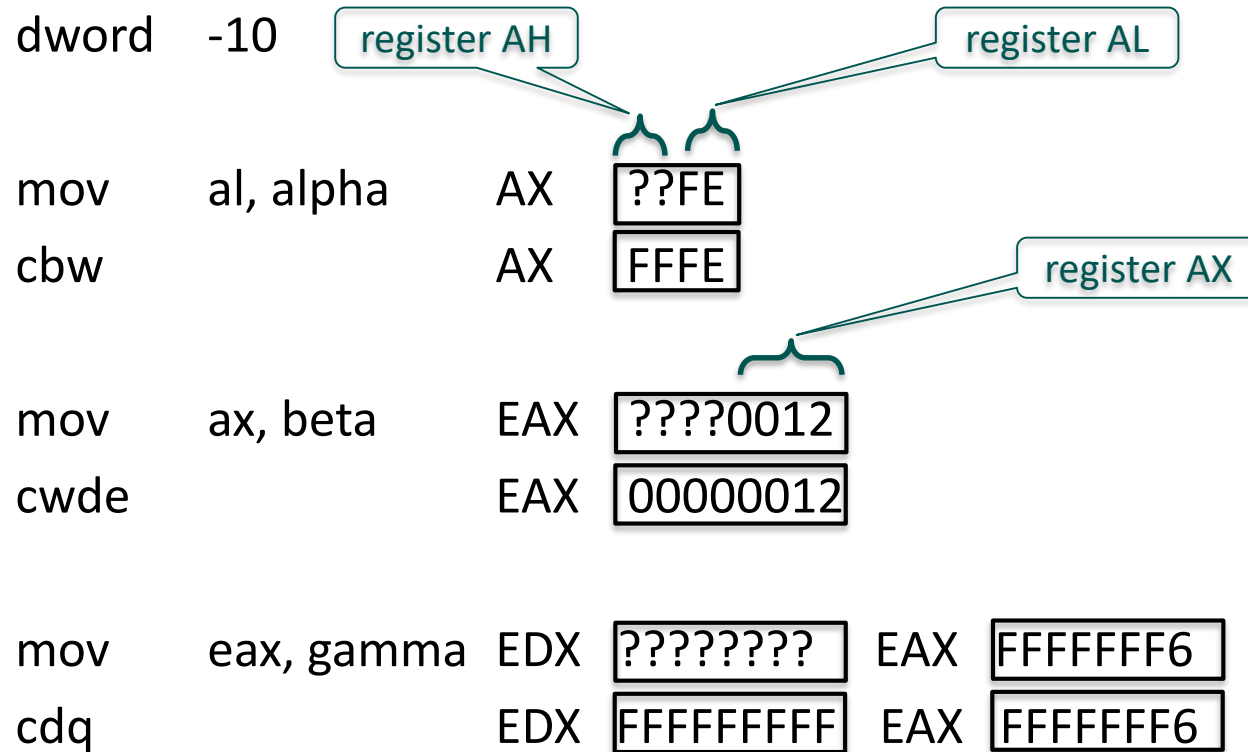
Instructions doubles the size of the source by means of sign extension:

- **CBW** (**convert byte to word**) copies the sign (bit 7) in the source operand into every bit in the register AH.
- **CWD** (**convert word to doubleword**) copies the sign (bit 15) of the word in the register AX into the 16 bits of the register DX.
- **CWDE** (**convert word to doubleword**) copies the sign (bit 15) of the word in the register AX into the high 16 bits of the register EAX.
- **CDQ** (doubleword to quadword) copies the sign (bit 31) of the doubleword in the register EAX into the 32 bits of the register EDX, forming the quadword EDX:EAX.

All these instructions have no operands and convert the contents of parts of the accumulator register EAX

Examples

alpha byte -2
 beta word 18
 gamma dword -10



Arithmetic instructions.

Single operand multiplication

MUL operand

IMUL operand

- Performs an unsigned (MUL) or signed (IMUL) multiplication of the *operand* with the register AL, AX or EAX (depending on the size of the *operand*) and stores the result in the register AX, DX:AX or EDX:EAX (depending on the size of the *operand*).
- Result size is always 2 times larger than multiplication operands size
- The *operand* can be a general-purpose register or a memory location. The action of this instruction and the location of the result depends on the opcode and the *operand* size:
 - AL * byte → AX
 - AX * word → DX:AX
 - EAX * dword → EDX:EAX

```
mov ax, 5  
mov bx, -2  
imul bx
```

; DX:AX = AX * BX

DX FFFF AX FFF6

Arithmetic instructions.

Two and three operand signed multiplication

IMUL operand1, operand2

- With this form the *operand1* is multiplied by the *operand2*. The *operand1* can be a general-purpose register only but the *operand2* can be an immediate value, a general-purpose register, or a memory location. The product is then stored in the *operand1*. Examples:
 - IMUL EBX, EAX → EBX = EBX * EAX
 - IMUL ECX, 5 → ECX = ECX * 5

IMUL operand1, operand2, operand3

- This form requires three operands. The *operand2* (can be a general-purpose register or a memory location) is multiplied by the *operand3* (can be an immediate value only). The product is then stored in the *operand1* (can be a general-purpose register only). Examples:
 - IMUL ECX, EAX, -6 → ECX = EAX * -6
 - IMUL ECX, TT, 5 → ECX = TT * 5
- With the two and three operand forms, the result is truncated to the size of the *operand1* register before it is stored in the *operand1*.

Arithmetic instructions.

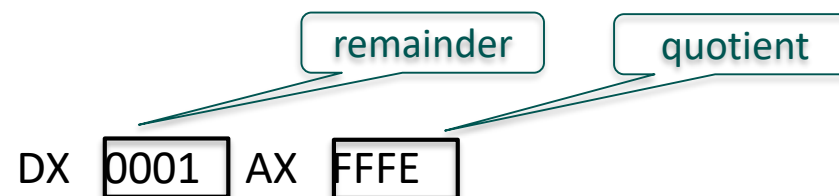
DIV

DIV operand

IDIV operand

- **Divides** unsigned (DIV) or signed (IDIV) **value in the AX, DX:AX, or EDX:EAX** (dividend, numerator) **by the source operand** (divisor, denominator) and stores the result in the AX (AH:AL), DX:AX, or EDX:EAX registers in a form *remainder:quotient*.
- The operation of dividing integers always has two results - the division (quotient) and the remainder.
- The dividend operand is always 2 times longer than each of the two results.
- The source operand can be a general-purpose register or a memory location. The action of this instruction depends on the operand size (dividend/divisor).
- Non-integral results are truncated (chopped) towards 0. The remainder is always less than the divisor in magnitude.

```
mov dx, 0
mov ax, 5
mov bx, -2
idiv bx ; DX:AX / BX
```



Simple calculations. Example

; Calculate formula $w = x/y - 3*z$ where x, y and z are word format integers
; Store the result in doubleword memory field w

.data

x word 17
y word 4
z word 5
w dword ?

.code

start proc

; compute x/y first

mov ax, x	; copy two bytes from memory address x into 16-bit register ax
cwd	; convert word to doubleword - extends ax to 32 bits in dx:ax
idiv y	; dx:ax / y -> quotient in ax and the remainder in dx
mov bx, ax	; save quotient in bx

; compute 3*z

mov ax, z	; copy two bytes from memory address z to ax
mov cl, 3	; set cl to value 3
imul cl	; ax = ax * cl

; compute result

sub bx, ax	; subtract, result in bx
movsx eax, bx	; copy bx with sign extension to eax
mov w, eax	; copy eax to memory field starting from the address w

; . . .

Thank You!

